Spring

Spring :

It is used for the dependency injection. In spring we will have a IOC Container which will store the objects and make them loosely coupled instead of hard coupling.

Hard Coupling :

If two or more components are dependent on each other then it is called as hard coupling.

Example :

Think that you are having a interface A and class B, now you implemented the A to B. While object creation we can use the parent reference object to assign the class B space. Here we use the new keyword which we are creating the hard object creation. Here the B is completely based on A.

IOC container :

Here we define which classes will have the objects then it will create the object to them and provides to us. It will look after the object like creating deleting…

Getting started with the spring :

1. *Create a simple maven project.*
2. *Go to maven repo and search for spring core*
3. *Copy and paste the dependency In pom.xml file.*

Syntax :

```
<dependencies>
    <dependency>
        Spring-core dependency
    </dependency>
</dependencies>
```

Example :

```
<dependencies>
    <!-- https://mvnrepository.com/artifact/org.springframework/spring-core -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-core</artifactId>
      <version>6.1.6</version>
    </dependency>
</dependencies>
```

4. *Also get the spring context from maven repo and place them in the dependencies.*

```
<!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
  <groupId>org.springframework</groupId>
   <artifactId>spring-context</artifactId>
   <version>6.1.6</version>
</dependency>
```

NOTE: It is always a best practice to take the same versions when using them.

Application Context in spring :

Application Context is the pillar for the spring as it takes care of the configuration and the dependency injection (Object creation A.K.A Beans).

1. To implement the application context we use the interface ApplicationContext.

   Syntax :

   ApplicationContext AppContextObject = new classPathXmlApplicationContext(“ TheBeansFile.xml”);

   Example :

```
ApplicationContext appContext = news
ClassPathXmlApplicationContext("Beans.xml");
```

ApplicationContext : It is an interface and it was implemented by the classPathXmlApplicationContext class.

2. Here we can mention that we can work with the xml or the annotations.
3. ***Configuration using XML :***
    a. *Go to resources*
    b. *Create a xml file.*
    c. *Go to google and search for spring xsd.*

Example :

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="
      http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

   <!-- bean definitions here -->

</beans>
```

    d. *Here we need to create a bean tag inside the beans.*
    Syntax :
    <bean id = “unique for Bean To Bean” class = “complete class path”></bean>

Example :

```
<bean id ="iphone" class = "com.CoreSpring.FirstBeans.Iphone"></bean>
   <bean id ="redmi" class = "com.CoreSpring.FirstBeans.Redmi"></bean>
```

NOTE : Writing the bean tag in xml will create a object for the class automatically.

    e. *To use the bean(Object) we have to send the id while calling for the bean. To get the bean we use the getBean() which will return the object class, If you want to work with the bean as object then we have to type convert the object.*
    Syntax :
    Object object = appContextObject.getBean(“id”);

Example :

```
Iphone iphone = (Iphone)appContext.getBean("iphone");
iphone.receiveMessage("Hello World");
```

    f. *If you don’t want to do the type casting then we have to send the another argument along with the id of bean.*
    Syntax :
    Object object = appContextObject.getBean(“id”, className.class);

Example :

```
Redmi redmi = appContext.getBean("redmi", Redmi.class);
redmi.receiveMessage("Hello Redmi");
```

NOTE : In Spring by default it follows the singleton design pattern. Means it will create a bean for once and it will be used multiple times.

Tight Coupling Example :

```
WhatsApp app1 = new Iphone();
WhatsApp app2 = new Redmi();

app1.sendMessage("Hello Iphone");
app2.sendMessage("Hello Redmi");
```

In the above example as you can see we are hard coding the object creation with the help of new and developer has to do it by himself. The app1 and app2 are tightly coupled to the object.

Loose Coupling Example :

```
WhatsApp app = appContext.getBean("iphone", WhatsApp.class);
app.receiveMessage("Hello Iphone");

app = appContext.getBean("redmi", WhatsApp.class);
app.receiveMessage("Hello Redmi");
```

Data Injection :

Here we inject the values from the xml instead of the using the constructor. As the spring is taking care of the Objects creation we can't use the constructor hence we have to pass the values to the spring to assign them.

Steps to follow for data injection for primitive Types :

1. To pass the values to the variable we have to write a set of properties inside the bean tag.
   Syntax :

```
<bean id = "uniqueValue" class = "classPath">
    <property name = "varName1" value = "value/Data"></property>
    <property name = "varName2" value = "value/Data"></property>
    .
    .
    .
    <property name = "varNamen" value = "value/Data"></property>
</bean>
```

   Example :

```
<bean id = "student" class="com.CoreSpring.FirstBeans.Student">
    <property name="id" value ="001"/>
    <property name="name" value ="Raj"/>
    <property name="marks" value="90.5"/>
</bean>
```

NOTE : The dependency injection will be invoked during the bean(object) creation. It will always looks for the setter method. Hence we have to use the setter method if not it will throw an error.

2. To pass the values to the variable we use another way which is from constructors. Here we use the constructor-arg and apart from it everything is same.
   Syntax :

```
<constructor-arg name = "uniqueValue" value = "value/Data"></constructor-arg>
.
.
.
<constructor-arg name = "unqiueValue" value = "data"> </constructor-arg>
```

   Example :

```
<bean id = "student" class="com.CoreSpring.FirstBeans.Student">
    <constructor-arg name="id" value ="001"/>
    <constructor-arg name="name" value ="Raj"/>
    <constructor-arg name="marks" value="90.5"/>
</bean>
```

NOTE: If you are using the constructor-arg we have to use the constructors to initialize the variables.

Steps to follow for data injection for Reference/Object Types :

1. If you want to initialize the reference object we have to create a bean for the reference class then we have to initialize it.
2. In property tag of the tag we use the ref property.
   Syntax :

```
<constructor-arg/ property name = "uniqueId"  ref = "beanName" />
```

   Example :

```
<bean id="student" class="com.CoreSpring.FirstBeans.Student">
        <constructor-arg name="id" value="001"/>
        <constructor-arg name="name" value="Raj"/>
        <constructor-arg name="marks" value="90.5"/>
        <constructor-arg name="address" ref="address" />
</bean>

<bean id="address" class="com.CoreSpring.FirstBeans.Address">
        <property name="street" value="No street"/>
        <property name="city" value="This city"/>
</bean>
```

NOTE : It will make hard if we forget to create a bean for the reference class then it will throw an error. Hence we use the auto wiring concept as it will be difficult to create bean for each and every class.

Auto-wiring :

It is going to bind the dependency objects automatically. Here we are going to mention the beans and the spring will take care of the reference by itself.

Syntax :

```
<bean id = “uniqueId” class = “qualifiedName” autowire = “type”>
</bean>
```

Here auto-wiring can be done in 3 ways.

1. ***byType*** :
    a. Spring will look for a bean of the same type required by the dependent bean and inject it. If there are multiple beans of the same type, Spring may throw an error unless you specify additional qualifiers.

    Syntax :

    ```
    <bean id = “uniqueId” class = “qualifiedName” autowire = “byType”>
        <property name = “varName” value = “data” />
    </bean>
    ```

    Example :

    ```
    <bean id="student" class="com.CoreSpring.FirstBeans.Student"
    autowire="byType">
        <property name="id" value="001"/>
        <property name="name" value="Raj"/>
        <property name="marks" value="90.5"/>
    </bean>

    <bean id="address" class="com.CoreSpring.FirstBeans.Address">
        <property name="street" value="No street"/>
        <property name="city" value="This city"/>
    </bean>
    ```

NOTE : To work with the byType autowire we have to use the property and also the name of the reference class has to be same type as mentioned in the bean or else it will throw exception.

2. ***Byname***
    a. Spring will look for a bean with the same name as the property or constructor argument and inject it.

    Syntax :

    ```
    <bean id = “uniqueId” class = “qualifiedName” autowire = “byname”>
        <property name= “varName” value = “data” />
    </bean>
    ```

    Example :

    ```
    <bean id="student" class="com.CoreSpring.FirstBeans.Student"
    autowire="byName">
        <property name="id" value="001"/>
        <property name="name" value="Raj"/>
    ```

```
        <property name="marks" value="90.5"/>
    </bean>

    <bean id="address" class="com.CoreSpring.FirstBeans.Address">
        <property name="street" value="No street"/>
        <property name="city" value="This city"/>
    </bean>
```

NOTE : The name of the reference bean has to be same as mentioned in the main bean.

3. **constructor**
    a. Spring will look for a constructor and automatically pass dependencies as arguments to that constructor.

    Syntax :

    ```
    <bean id = “uniqueId” class = “qualifiedName” autowire = “constructor”>
        <constructor-arg name = “varName” value = “data” />
    </bean>
    ```

    Example :

    ```
    <bean id="student" class="com.CoreSpring.FirstBeans.Student"
    autowire="constructor">
        <constructor-arg name="id" value="001"/>
        <constructor-arg name="name" value="Raj"/>
        <constructor-arg name="marks" value="90.5"/>
    </bean>

    <bean id="address" class="com.CoreSpring.FirstBeans.Address">
        <property name="street" value="No street"/>
        <property name="city" value="This city"/>
    </bean>
    ```

NOTE : To work with the constructor we have to use the constructor-arg or else it will not work.

NOTE : It must contain the getters and setters method.

Steps to follow for data injection using annotations :

1. To use the autowire in java we can use the annotation @autowire.
2. Generally we have to activate the support for the autowire.
3. To activate it we have to go to xsd file. We have to use the annotation config file with it.
    Example :

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:context="http://www.springframework.org/schema/context" xsi:schemaLocation="
      http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
      http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd"> <!-- bean definitions
here -->

</beans>
```

4. We have to enable the annotation support. By default the annotation support is disabled. To enable use the tag <context:annotation-config></context : annotation-config>
    Syntax :

    ```
    <context:annotation-config></context : annotation-config>
    ```

    Example :

    ```
    <context:annotation-config></context:annotation-config>
    ```

5. We have to use the autowire annotation at the setter side.
    Syntax :

    ```
    @Autowired
    Public void setMethod(){
    ```

```
        //block of code
    }
```

Example :

```
@Autowired
public void setAddress(Address address) {
    this.address = address;
}
```

6. When you are having more than one bean then we will get an exception as the autowired can use only one bean to auto-configure. To remove this ambiguity we use the annotation qualifier.

Syntax :

```
@Autowired
@Qualifier("beanId")
Public void setterMethod(){
    //block of code
}
```

NOTE : Autowired will look for byType by default, if there is any ambiguity then it will look for byName.

NOTE : If your class is containing the variables and setters and getters then it is called as POJO (plain old java object)

Data Injection for the collection :

To insert the data into the collection we have to use the property tag and inside it we have to use the collection tag and then go for the value tag.

Syntax :

```
<property name = "varName">
    <list>
        <value>data</value>
        <value>data</value>
        .
        .
    </list>
</property>
```

Example :

```
<property name = "hobbies">
    <list>
        <value>Cricket</value>
        <value>Chess</value>
        <value>dance</value>
    </list>
</property>
```

Note : For every collection we will have the tag in xml. There is a slight minor thing which we need to use for the map.

Syntax :

```
<property name = "value">
    <map>
        <entry key = "keyName" value ="data"></entry>
        <entry key = "keyName" value ="data"></entry>
        <entry key = "keyName" value ="data"></entry>
    </map>
</property>
```

Example :

```
<property name = "hobbies">
    <map>
        <entry key = "key1" value = "4"></entry>
        <entry key = "key2" value = "42"></entry>
        <entry key = "key3" value = "44"></entry>
```

```
        </map>
    </property>
```

NOTE : To insert the reference we use the value-ref property.

TIP : We have to externalize the xml code to make it more independent and developer friendly. To do that we have to create a file called application and need to save it with the extension properties.

1. To do that create a application.properties file.
2. We use the Key, values to store the data into application.properties.
    a. **Syntax :**
        Key = value
    Example :

```
        student.id = 1
    student.name = "Raj Raj"
```

3. Use the property placeHolders to initialize the values.
    a. **Syntax** :
        ${key}
4. But the thing is to activate it we have to use the context:property-placeholder to externalize the data.
    a. **Syntax** :
        <context:property-placeholder location = “classpath : filename” />
    Example :
        <context:property-placeholder location =
        "classpath:application.properties"/>

NOTE: We have to save the file inside the resources folder.

5. To use the place holder we are using the ${value}.
    a. **Syntax** :
        <property name = “variableName” value = “${key}”></property>
    Example :
        <property name="name" value="${student.name}"/>

No XML Configuration :

From now on we will use the annotations to work with spring.

1. To work with the annotation we have use the context:property-placeholder
2. To inject values into the variables we use the @Value annotation.
    a. **Syntax** :
        @Value(“data”)
        Private dataType varName;
    Example :

```
        @Value("34")
        private int id;
        @Value("Teks")
        private String name;
```

3. We are going to use the @Component to create a bean automatically. Instead of the
    a. **Syntax** :
        @Component
        Public class ClassName{
            //class contents
        }
    Example :

```
        @Component
    public class Student {
```

4. To mention that we are having beans in the project we need to use the component-scan tag.
    a. **Syntax** :

<context:component-scan base-package =
“packageName”></context:component-scan>

Example :

```
<context:component-scan base-
package="com.CoreSpring.FirstBeans"></context:component-scan>
```

NOTE : When you are using the @Component annotation it will create a default bean with the same name as className but in lower characters. If you want to specify the id then we have to mention the name after the annotation.

Syntax :

```
@Component( “idName” )
Public class ClassName{
    //body of the class
}
```

Example :

```
@Component("Student")
public class Student {
  //body of the class
}
```

A brief steps for the no xml configuration :

1. In xml we need to use the below tags
   a. Use the <context:annotation-config></context:annotation-config> to enable the annotation.
   b. Use the <context:property-placeholder></context:property-placeholder> to enable the externalize the values
   c. Use the <context:component-scan base-package=”parentPackage”></context:component-scan>

Creating the spring application without xml file :

1. We need to change the ClassPathXmlApplicationConfiguration to the annotation base.
   a. **Syntax** :

```
ApplicationContext contextObject = new
AnnotationConfigApplicationContext( configurationClass.class );
```

   Example :

```
ApplicationContext appContext = new
AnnotationConfigApplicationContext(ConfigurationFile.class);
```

2. As the xml file is actually acting as the configuration file and we are removing it hence it will throw an exception.
3. To remove the exception and to carry with the configuration we have to create a new class file and we have to use the annotation to it.
   a. **Syntax** :

```
@Configuration
Public class ClassName{
    //body of the class
}
```

4. Now we have to apply the configurations here.
   a. To scan the components we use the @ComponentsScan annotation.
      i. **Syntax** :

```
@Configuration
@ComponentScan( basePackages = “basePackage” )
Public class className{
    //body of the class
}
```

      Example :

```
@Configuration
@ComponentScan( basePackages = "com.CoreSpring")
public class ConfigurationFile {
```

```
}
```

b. To use the externalize values we use the PropertySource(“classpath : .propertiesName”).

i. **Syntax** :

```
@Configuration
@ComponentScan( basePackages = “basePackage”)
@PropertySource( “classpath: propertyName” )
Public class className{
        //body of the class
}
```

Example :

```
@Configuration
@ComponentScan( basePackages = "com.CoreSpring")
@PropertySource("classpath:application.properties")
public class ConfigurationFile {

}
```

5. Using the autowired annotation to link the address.

a. **Syntax** :

```
@Autowired
@Qualifier( “Name”) //optional
Private type ReferenceObject;
```

Example :

```
@Value("${student.id}")
private int id;
@Value("${student.name}")
private String name;

@Autowired
@Qualifier("address")
private Address address;
```

6. If you want to have a custom object we are going to use the @Bean. Here we are creating the object but the spring will take care of the dependency injection.

a. **Syntax** :

```
@Bean
Public ClassName methodName{
        Return new constructor( );
}
```

Example :

```
@Bean
public Student getStudent() {
        return new Student();
}
```

NOTE : When you are using the @Bean there is no requirement for using the @Component annotation. To create a bean we have to use the methodName as bean id.

Example :

```
Student student = appContext.getBean("getStudent", Student.class);
```

NOTE : You create your own bean id using the name property.

Syntax :

```
@Bean( name = “value”)
Method( ){
        //body
}
```

Example for configuration :

```
@Bean(name = "student")
```

```
public Student getStudent() {
        return new Student();
}
```

Example for mainMethod :

```
Student student = appContext.getBean("student", Student.class);
```

NOTE : We can give multiple names to the bean with a curly brace.

Syntax :

```
@Bean(name = { “name1”, “name2”} )
Method(){
        //body
}
```

Example :

```
@Bean(name = {"student", "student1"})
public Student getStudent() {
        return new Student();
}
```

NOTE : We are going to two ways to create beans

1. ***BeanFactory*** : lazy loading
2. ***ApplicationContext*** : Eager Loading

JDBC using Spring :

Normal JDBC steps :

It follows the same steps as JDBC.

1. *Loading driver*
2. *Registering driver*
3. *Connecting to database*
4. *Creating statement*
5. *Executing query*
6. *Result set will generate*

Steps to work with the JDBC :

1. Create a maven project
2. We need to get the below dependencies.
    a. *Get the spring core and spring context*
    b. *Spring JDBC*
    c. *MySQL Connector*
3. Here we require two classes to work with the JDBC.
    a. ***Driver Manager Data Source :***
        i. Contains the database information.
        ii. It will have four values which we needs to provide
            1. *URL*
            2. *UserName*
            3. *Password*
            4. *DriverType*

Example :

```
<bean id = "dmds" class =
"org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value = "com.mysql.cj.jdbc.Driver"></property>
<property name="url" value = "jdbc:mysql://localhost:3306/tcs"></property>
<property name ="userName" value="root"></property>
<property name="password" value="0000"></property>
</bean>
```

    b. ***Spring JDBC template :***

i. Uses the DriverManagerDataSource

Example :

```
<bean id ="jdbcTemplate" class = "org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="dmds"></property>
</bean>
```

4. Now we need to connect the database but the spring will take of it. We have to getBean of the JdbcTemplate.
   Syntax :
   ApplicationContext appContextObject = new ClassPathXmlApplicationContext(“xmlFileName”);
   JdbcTemplate template = appContextObject.getBean(“Id/Name”, className.class);
   Example :

```
ApplicationContext appContext = new
ClassPathXmlApplicationContext("beanJDBC.xml");
JdbcTemplate template = appContext.getBean("jdbcTemplate", JdbcTemplate.class);
```

5. To insert data into the database we use the update method.
   a. **Syntax** :
      templateObject.update(“SQL”);
      Example :

```
int rows = template.update("Delete from student where id = 6");
```

CRUD Operations :

1. For create, update, delete we use the update method.
2. ***For Create :***
   a. We use the insert command to create.
   b. **Example** :

```
int rows = template.update("insert into student values(6, 'Stream')");
```

3. ***For Update :***
   a. We use the update command to update the table.
   b. **Example** :

```
int rows = template.update("update student set name = 'devil' where id = 3");
```

4. ***For Delete :***
   a. We use the delete command to delete the content from the table.
   b. **Example** :

```
int rows = template.update("Delete from student where id = 6");
```

5. ***For Read :***
   a. For read we need to catch either a single record or a multiple set of records. To handle it we are having two methods.
      i. jdbcTemplate.queryforobject (“SQL Query”, rowmapper); - returns single object
      ii. jdbcTempalte.query(“SQL_Query”, rowMapper) – return a list of objects

NOTE : Why do we need the row mapper to map the data in the database.

b. Creating Row Mapper :

i. To create row mapper we have to create a class.
ii. Implement the RowMapper interface to the class.
Syntax :

```
Classname implements RowMapper<mapperClass>{
    //body of the class
}
```

iii. It will have an implementation method which will contain the result set parameter and rowNumber.
iv. We have to use the result set methods to retrieve the data.

NOTE : The template.update will return the no.of rows that were effected

i. Jdbc Template QueryForObject :

Example :

```
RowMapper rowMapper = new RowMapper() {
```

```
                @Override
                public Student mapRow(ResultSet resultSet, int rows) {
                        Student student = new Student();
                        System.out.println("inside the mapRow method");
                        try {
                                int id = resultSet.getInt("id");
                                String name = resultSet.getString("name");
                                System.out.println(id +" " + name);
                                student.setId(id);
                                student.setName(name);
                                System.out.println(student);
                        } catch (SQLException e) {
                                // TODO Auto-generated catch block
                                e.printStackTrace();
                        }
                        return student;
                }
        };

        Student student = template.queryForObject("select * from student where
id = 7", rowMapper);
        System.out.println(student);
```

ii. jdbc Tempate Query :
Example :

```
                RowMapper rowMapper = new RowMapper() {
                @Override
                public Student mapRow(ResultSet resultSet, int rows) {
                        Student student = new Student();
                        System.out.println("inside the mapRow method");
                        try {
                                int id = resultSet.getInt("id");
                                String name = resultSet.getString("name");
                                System.out.println(id +" " + name);
                                student.setId(id);
                                student.setName(name);
                                System.out.println(student);
                        } catch (SQLException e) {
                                // TODO Auto-generated catch block
                                e.printStackTrace();
                        }
                        return student;
                }
        };

            ArrayList<Student> students = new ArrayList<Student>();
        students =  (ArrayList<Student>) template.query("select * from student", rowMapper);
        System.out.println(students);
```